Carbon Cycle Numerical Model: Implementation Report

March 10, 2026

1 Scope

This report documents what is implemented in `carbone.py` and how it is implemented:

- an 8-state carbon-cycle ODE model,
- a forward-Euler time integration scheme,
- post-processing and plotting with per-curve min–max normalization,
- automatic PDF naming with encoded initial conditions, horizon, and time step.

2 State, Parameters, and Forcing

The state vector is ordered as

$$\boldsymbol{x}(t) = \begin{bmatrix} A & R & D & F & P & S & O & V \end{bmatrix}^\top,$$

where:

- A: atmosphere carbon,
- R: carbonate rock carbon,
- D: deep ocean carbon,
- F: fossil-fuel carbon stock,
- P: plant carbon,
- S: soil carbon,
- O: surface ocean carbon,
- V: vegetated land area (%).

Useful constants (from the script) are:

$$\alpha_A = \frac{280}{A_0}, \quad k_P = \frac{55}{P_0}, \quad k_S = \frac{55}{S_0}, \quad k_{sed} = \frac{0.1}{D_0}, \quad k_{down} = \frac{90.1}{O_0}, \quad k_{up} = \frac{90}{D_0}.$$

With deforestation parameter Def:

$$\beta = \frac{\text{Def}}{2}, \qquad \delta = \left(\frac{\text{Def}}{P_0} \cdot 0.2\right) 100, \qquad \nu = 0.1 \text{ (volcanic flux)}.$$

The fossil-combustion forcing is piecewise linear in time from tabulated data:

$$\mathcal{F}_{comb}(t) = \text{linear interpolation of } (t_i, f_i),$$

and applied with stock saturation logic:

$$u_F(t, F) = \begin{cases} \mathcal{F}_{comb}(t), & F > 0, \\ 0, & F \leq 0. \end{cases}$$

3 ODE System in Matrix Form (Linear + Nonlinear Split)

The implemented system can be written as

$$\dot{\boldsymbol{x}} = A_{\text{lin}}\,\boldsymbol{x} + \boldsymbol{b} + \boldsymbol{g}(\boldsymbol{x}, t).$$

Linear part

$$A_{\text{lin}} = \begin{bmatrix} -\text{Kao}\,\alpha_A & 0 & 0 & 0 & k_P & k_S & 0 & 0 \\ 0 & 0 & k_{sed} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -(k_{sed}+k_{up}) & 0 & 0 & 0 & k_{down} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -2k_P & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & k_P & -k_S & 0 & 0 \\ \text{Kao}\,\alpha_A & 0 & k_{up} & 0 & 0 & 0 & -k_{down} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix},$$

$$\boldsymbol{b} = \begin{bmatrix} \beta+\nu \\ -\nu \\ 0 \\ 0 \\ -2\beta \\ \beta \\ 0 \\ -\delta \end{bmatrix}.$$

Nonlinear / time-dependent part

$$\boldsymbol{g}(\boldsymbol{x}, t) = \begin{bmatrix} \nu_F(t,F) + \text{Kao}\,pCO_2^{oc}(A,O) - \Phi(A,V) \\ 0 \\ 0 \\ -\nu_F(t,F) \\ \Phi(A,V) \\ 0 \\ -\text{Kao}\,pCO_2^{oc}(A,O) \\ 0 \end{bmatrix}.$$

Where:

- $\Phi(A,V)$ is photosynthesis,

$$\Phi(A,V) = 110 \cdot CO2Effect(\alpha_A A) \cdot \frac{V}{100} \cdot TempEffect(GlobalTemp(\alpha_A A))\,,$$

- $pCO_2^{oc}(A,O)$ is the ocean partial pressure chain

  $A \to GlobalTemp \to WaterTemp \to (K_{CO_2}, K_{carb}), \quad O \to SurfCConc \to HCO_3^- \to CO_3^{2-} \to pCO_2^{oc},$

  which is nonlinear (rational terms and square root in carbonate chemistry).

Hence, the model is **affine-linear plus nonlinear closure/forcing**. The main nonlinearities are $\Phi(A,V)$ and $pCO_2^{oc}(A,O)$, plus the piecewise term $\nu_F(t,F)$.

4 Numerical Time Integration

The solver is explicit Euler:

$$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \Delta t\, \boldsymbol{f}(\boldsymbol{x}_n, t_n),$$

with time grid generated by

$$t_n = t_0 + n\Delta t, \quad n = 0, \ldots, N, \quad t_N \approx t_f.$$

In code:

- `derivative(x,t)` computes $\boldsymbol{f}(\boldsymbol{x}, t)$,
- `step(x,t,dt)` applies one Euler step,
- `run_simulation(...)` iterates over all steps and stores trajectories.

5 Plotting and Output Encoding

Implemented plotting features:

- 3 active subplots in a 2×2 canvas (top-left, top-right, bottom-left),
- per-curve min–max normalization

  $$y_{norm} = \frac{y - y_{\min}}{y_{\max} - y_{\min}},$$

  with constant series mapped to 0.5,
- legends include each curve's physical range $[y_{\min}, y_{\max}]$ and use text color matching each curve,
- PDF filename encodes initial conditions, simulated years, and Δt in scientific notation-safe tags.

6 Reproducibility Notes

The report corresponds to the current `carbone.py` structure and parameterization (including current deforestation value in the script). If parameters or flux functions change, update the matrix split accordingly.